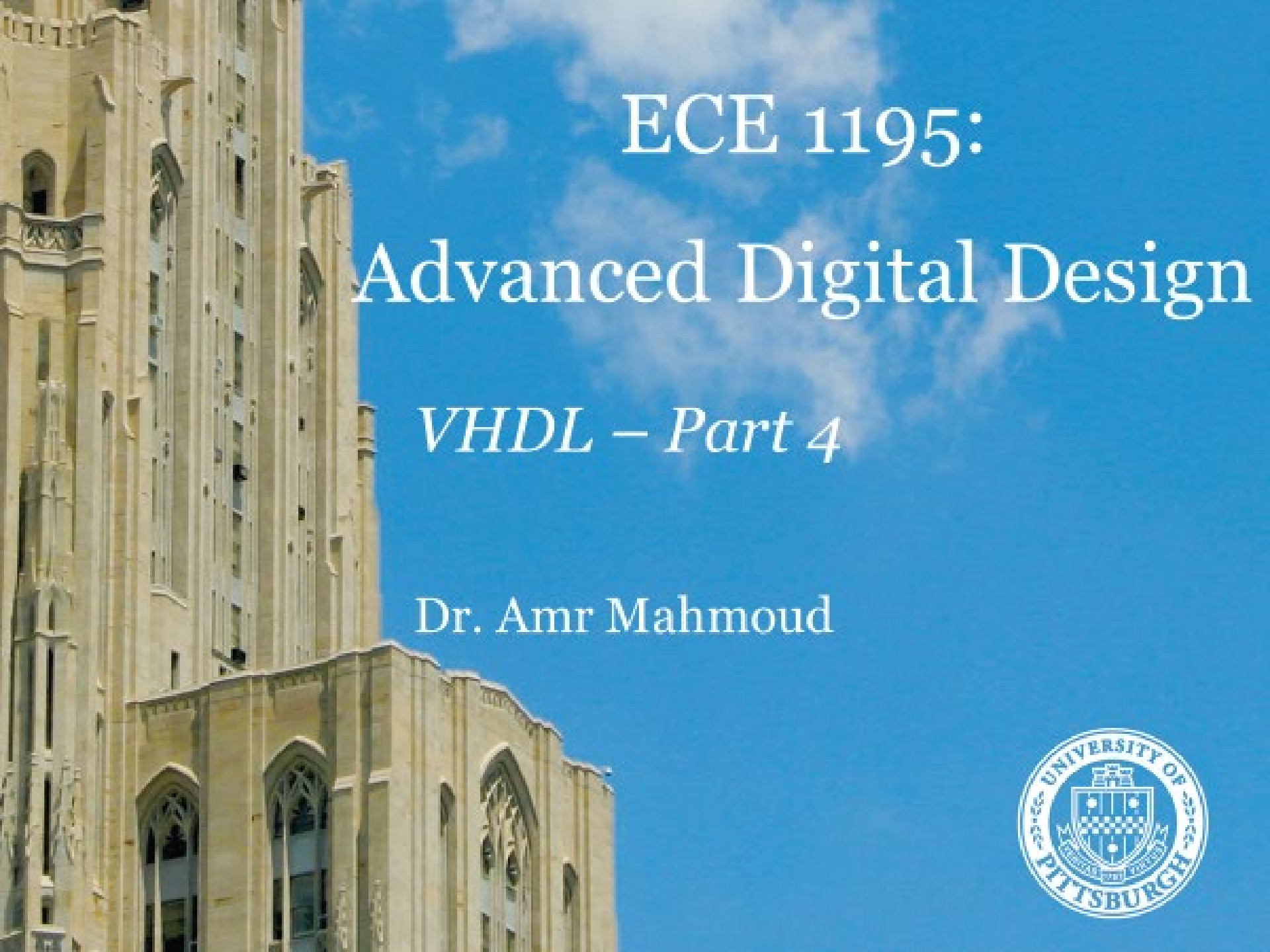


ECE 1195: Advanced Digital Design

VHDL – Part 4

Dr. Amr Mahmoud

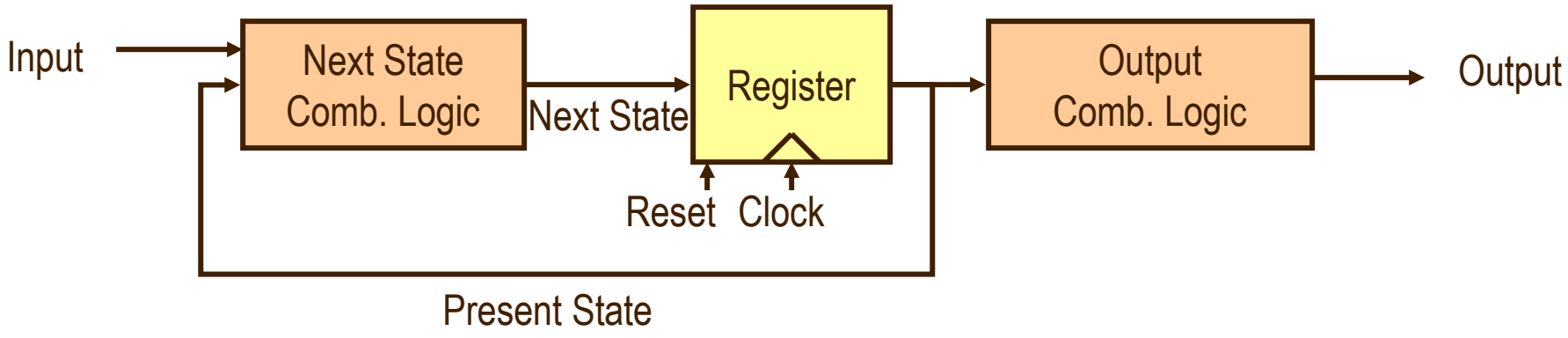
Behavioral VHDL Coding

Finite State Machine

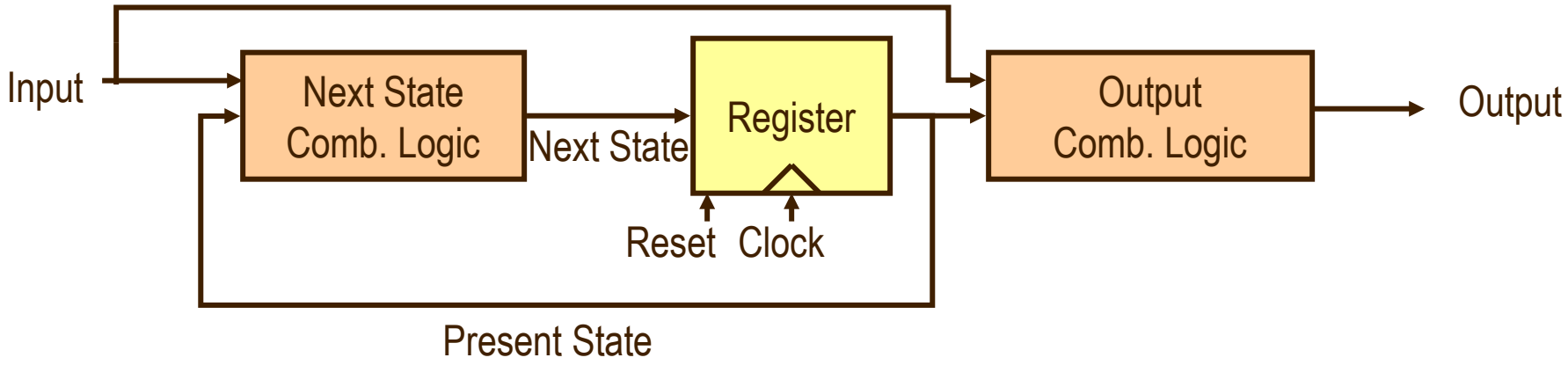
Agenda

- **Finite State Machine**
  - Moore Template
  - Mealy Template
  - Algorithmic State Machines.

Moore versus Mealy Machines

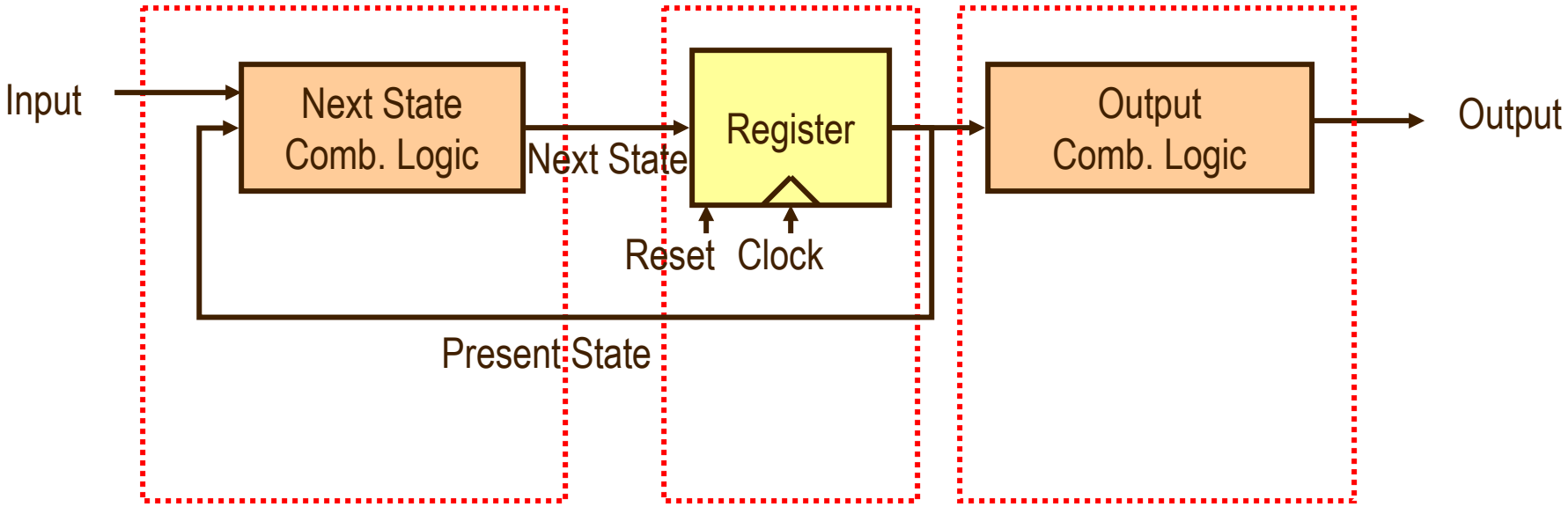


Moore Machine: **Next State = Function(Input, Present State),** **Output = Function(Present State)**



Mealy Machine: **Next State = Function(Input, Present State),** **Output = Function(Present State, Input)**

Moore Machines



NEXT STATE FUNCTION:

process(input, present state)

FSM REGISTER:

process(reset, clock)

OUTPUT FUNCTION:

process(present state) OR use concurrent code

Template for Moore Machines

```
architecture behavioral of fsm_example  is
  type state is (S0, S1, S2, S3); -- define the state type
  signal pr_state, nx_state: state;--declare present state & next state
begin
    -- section 1: fsm register
    process (resetn,clock)
    begin
        if (resetn='0') then
            pr_state <= S0; -- choose reset state
        elsif (clock'event and clock='1') then
            pr_state <= nx_state;
        end if;
    end process;
```

Template for Moore Machines(Cont.)

```
-- section 2: next state function
process (input, pr_state) -- or can use concurrent statements
begin
    case pr_state is
        when S0 =>
            if (input = . . . ) then
                nx_state <= S1;
            else
                nx_state <= S2;
            end if;
        . . .
    end case;
end process;
```

Template for Moore Machines(Cont.)

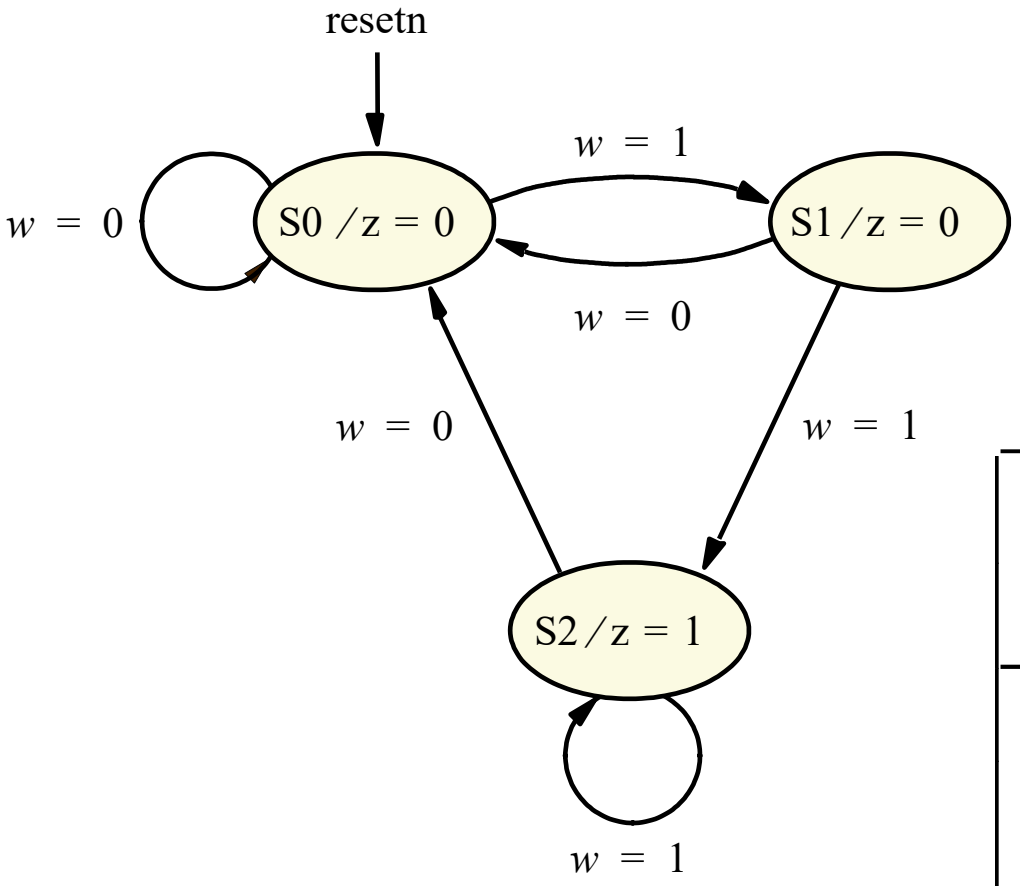
```
-- section 3: Output function
process (pr_state)-- or can use concurrent statements
begin
    case pr_state is
        when S0 =>
                Output <= . . . .;

        when S1 =>
                Output <= . . . .;
        . . .
    end case;
end process;
```

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are ‘1’s.**

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**



State Transition Table

Present state	Next state		Output z
	w = 0	w = 1	
S0	S0	S1	0
S1	S0	S2	0
S2	S0	S2	1

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity moore_fsm is
   port(w : in STD_LOGIC;
     clock : in STD_LOGIC;
     resetn : in STD_LOGIC;
     z : out STD_LOGIC);
end moore_fsm;
architecture behavioral of moore_fsm   is
  -- Signal Declaration
  type state is (S0, S1, S2);
  signal pr_state, nx_state: state;
begin
-- section 1: fsm register
process (resetn,clock)
begin
    if (resetn='0') then
      pr_state <= S0; -- reset state
    elsif (clock'event and clock='1') then
      pr_state <= nx_state;
    end if;
end process;
```

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**

```
-- section 2: next state function
process (w, pr_state) -- sensitive to both input and present state
begin
  case pr_state is
    when S0 =>
      if (w = '0') then
        nx_state <= S0;
      else
        nx_state <= S1;
      end if;
    when S1 =>
      if (w = '0') then
        nx_state <= S0;
      else
        nx_state <= S2;
      end if;
    when S2 =>
      if (w = '0') then
        nx_state <= S0;
      else
        nx_state <= S2;
      end if;
  end case;
end process;
```

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**

```
 -- section 3: output function
process(pr_state) -- sensitive to present state only
begin
   if pr_state = S2 then
         z <='1'; else
         z<='0';
    end if;
end process;
-- Also, you can concurrent statements
--z <= '1' when pr_state = S2 else '0';

end behavioral;
```

Alternative: Merging sections 2 & 3

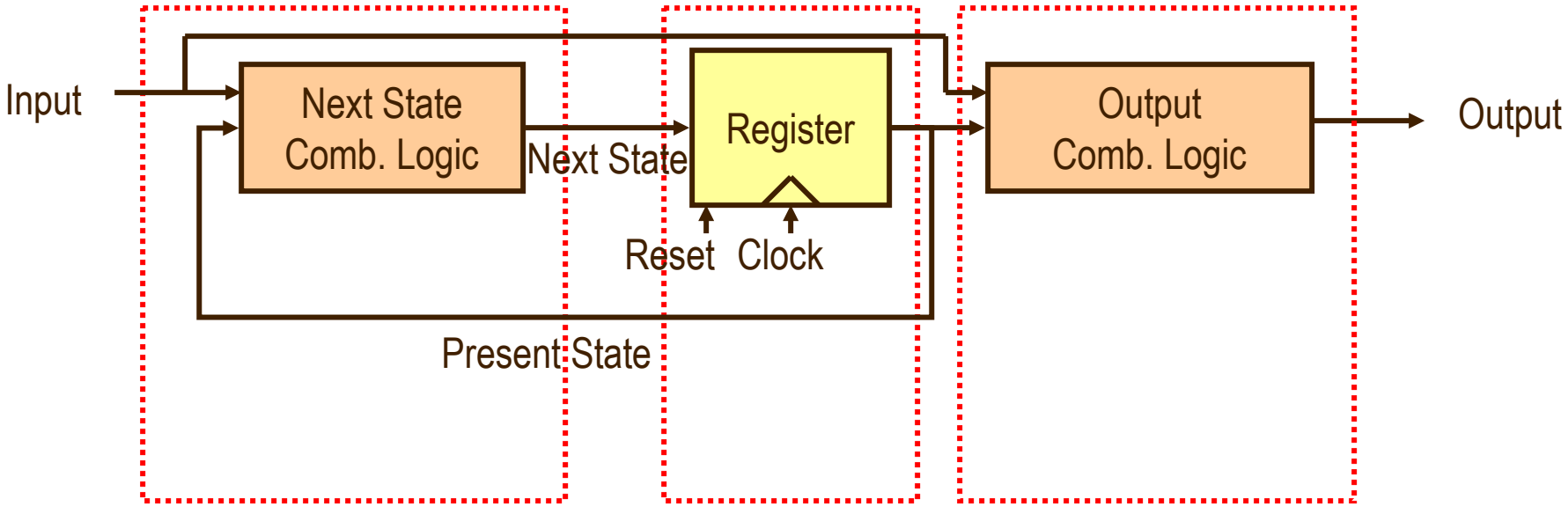
- Can also merge section 2 (next state function) and section 3 (output function)

```
-- section 2: next state function
process (w, pr_state) -- sensitive to both input and present state
begin
  case pr_state is
    when S0 =>
      Z <= '0';-- output assignment
      if (w = '0') then
        nx_state <= S0;
      else
        nx_state <= S1;
      end if;
    when S1 =>
      Z <= '0';
      if (w = '0') then
        nx_state <= S0;
      else
        nx_state <= S2;
      end if;
    when S2 =>
      Z <= '0';
      if (w = '0') then
        nx_state <= S0;
      else
        nx_state <= S2;
      end if;
  end case;
end process;
```

z<='0' goes before the IF statement.

Coding for Mealy Machines

Mealy Machines



NEXT STATE FUNCTION:

process(input, present state)

FSM REGISTER:

process(reset, clock)

OUTPUT FUNCTION:

process(input, present state) OR use concurrent code

Difference between Moore and Mealy is sensitivity to input

Template for Mealy Machines

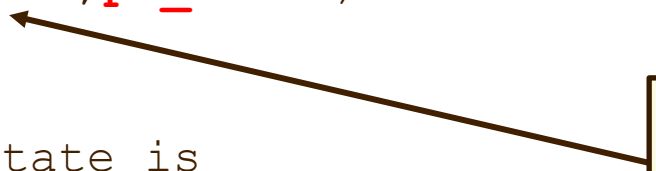
```
architecture behavioral of fsm_example  is
  type state is (S0, S1, S2, S3); -- define the state type
  signal pr_state, nx_state: state;--declare present state & next state
begin
    -- section 1: fsm register
    process (resetn,clock)
    begin
        if (resetn='0') then
            pr_state <= S0; -- choose reset state
        elsif (clock'event and clock='1') then
            pr_state <= nx_state;
        end if;
    end process;
```

Template for Mealy Machines(Cont.)

```
-- section 2: next state function
process (input, pr_state) -- or can use concurrent statements
begin
    case pr_state is
        when S0 =>
            if (input = . . . ) then
                nx_state <= S1;
            else
                nx_state <= S2;
            end if;
        . . .
    end case;
end process;
```

Template for Mealy Machines(Cont.)

```
-- section 3: Output function
process (input ,pr_state)-- or can use concurrent statements
begin
    case pr_state is
        when S0 =>
            if (input = . . . ) then
                output <= . . .;
            else
                output <= . . .;
            end if;
        . . .
    end case;
end process;
```

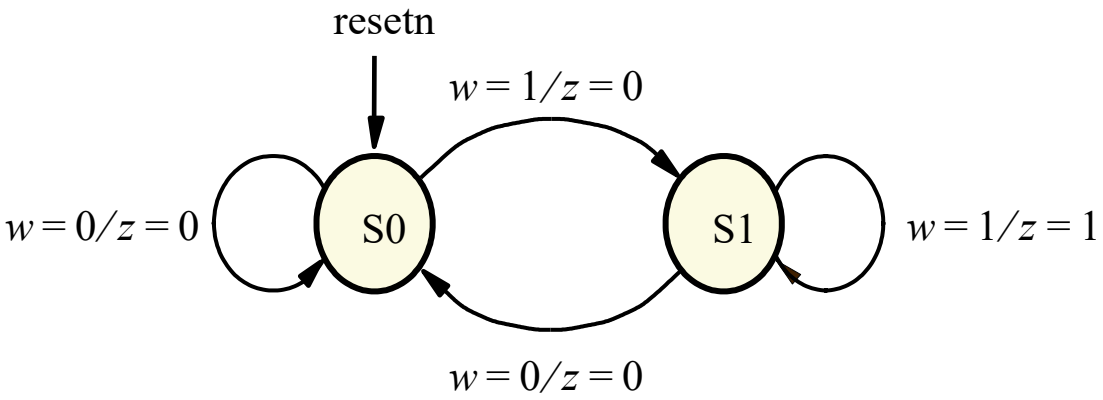


Difference between Moore and Mealy is in the output function (Mealy depends on input)

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are ‘1’s.**

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**



State Transition Table

Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
S0	S0	S1	0	0
S1	S0	S1	0	1

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity mealy_fsm is
   port(w : in STD_LOGIC;
     clock : in STD_LOGIC;
     resetn : in STD_LOGIC;
     z : out STD_LOGIC);
end mealy_fsm;
architecture behavioral of mealy_fsm   is
  type state is (S0, S1);
  signal pr_state, nx_state: state;
begin
  -- section 1: fsm register
  process (resetn,clock)
  begin
    if (resetn='0') then
      pr_state <= S0; -- reset state
    elsif (clock'event and clock='1') then
      pr_state <= nx_state;
    end if;
  end process;
```

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**

```
-- section 2: next state function
  process (w, pr_state)
  begin
    case pr_state is
      when S0 =>
        --if (w = '0') then
        --  nx_state <= S0;
        --else
        if (w = '  1'   ) then
          nx_state <= S1;
        end if;
      when S1 =>
        if (w = '0') then
          nx_state <= S0;
        --else
        --  nx_state <= S1;
        end if;
    end case;
  end process;
```

Solution

- Write the VHDL code that describe **the FSM that detects if two consecutive inputs are '1's.**

```
    -- Section 3: output function
    process (pr_state, w)
    begin
        if pr_state = S1 then
            if w = '1' then
                z <= '1';
            else
                z <= '0';
            end if;
        else
            z <= '0';
        end if;
    end process;

end behavioral;
```

Alternative: Merging sections 2 & 3

- Can also merge section 2 (next state function) and section 3 (output function)

```
-- sections 2 & 3: next state function, output function
process (w, pr_state) begin
  case pr_state is
    when S0 =>
      if (w = '0') then
        nx_state <= S0;
        z <= '0';
      else
        nx_state <= S1;
        z <= '0';
      end if;
    when S1 =>
      if (w = '0') then
        nx_state <= S0;
        z <= '0';
      else
        nx_state <= S1;
        z <= '1';
      end if;
  end case;
end process;
end behavioral;
```

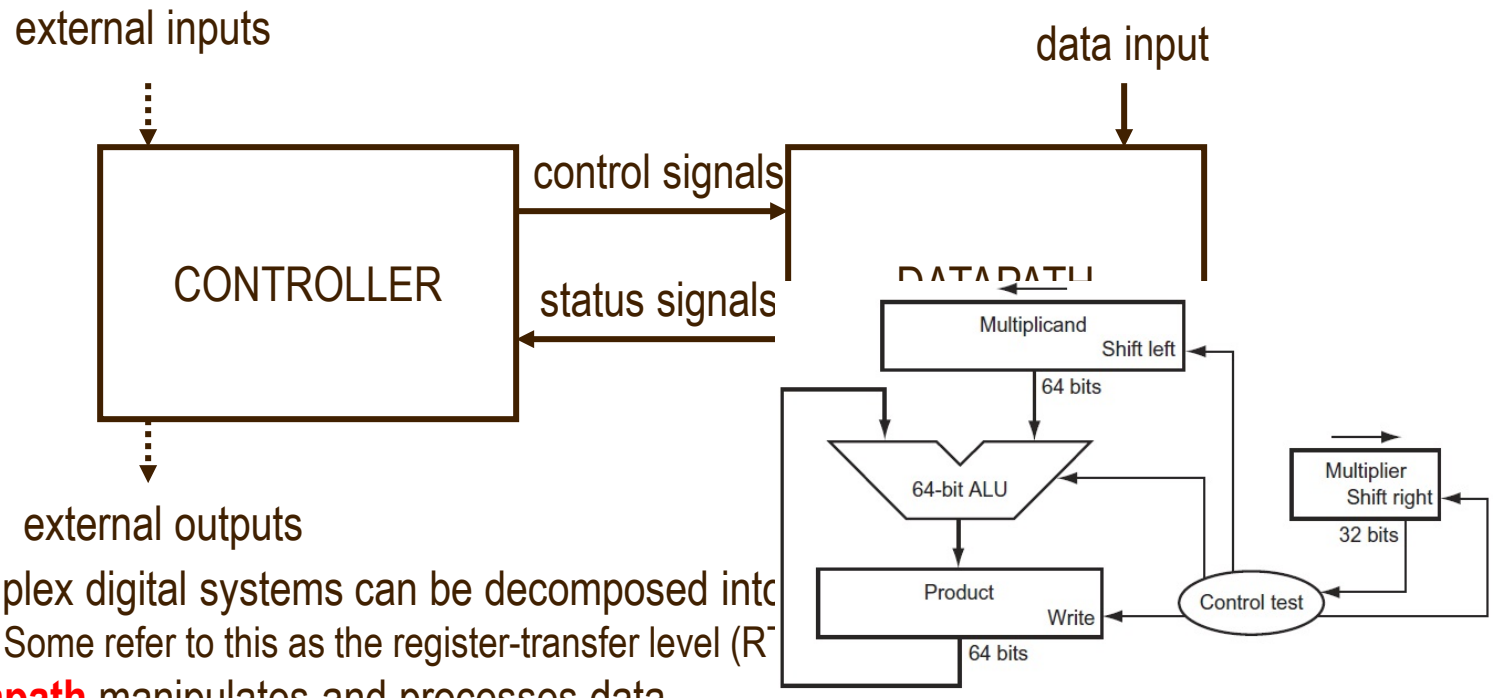
Normally, Mealy output goes inside the if statement

VHDL Coding: Mealy versus Moore Summary

- **Code in three sections**
  - Section 1: FSM register (same for Mealy, Moore)
  - Section 2: Next state function (same for Mealy, Moore)
  - Section 3: Output function
    - Code using concurrent statements or process statement
    - Moore depends on present state only
    - Mealy depends on present state and input
- **Can merge sections 2 and 3 together**
  - Moore assigns the output **before** the if statement
  - Mealy assigns the output **inside** the if statement

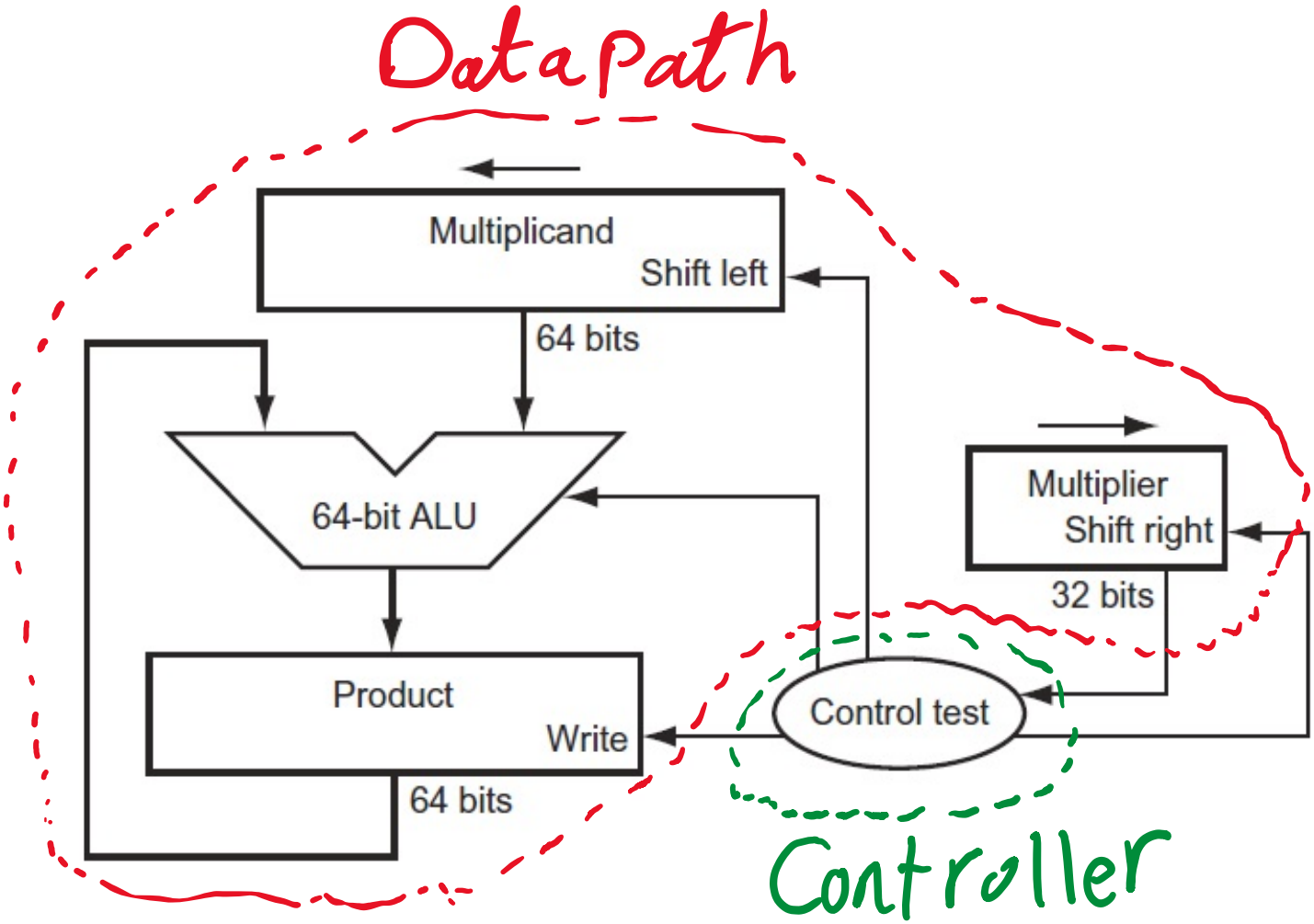
VHDL Coding for Algorithmic State Machines

Complex Digital System Design

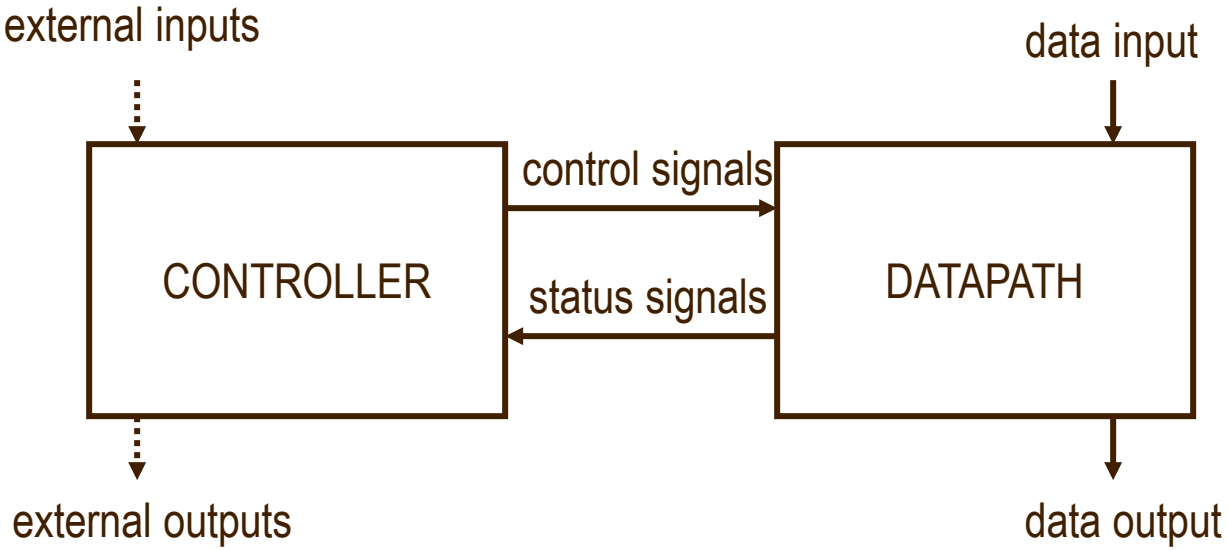


- Complex digital systems can be decomposed into
  - Some refer to this as the register-transfer level (RT
- **Datapath** manipulates and processes data
  - To perform arithmetic, logic, shifting, and other data-processing tasks
  - These operations are implemented with ALUs, registers, multiplexers, adders, comparators, etc.
- **Controller** determines the enabling and sequencing of Datapath operations
  - Controller provides signal to activate various processing in the Datapath
    - Example: enable signals for registers
    - Example: control signals for muxes
  - Controller also determines the sequence the operations are performed

Like Lab 3 Design

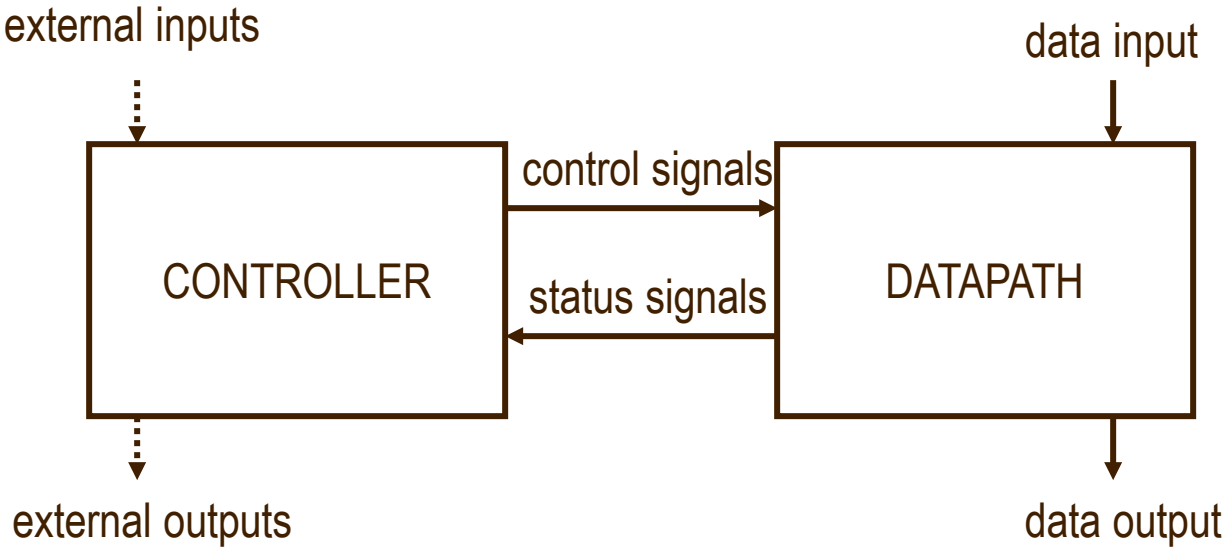


Inputs and Outputs



- Datapath inputs and outputs
  - **External data input**
    - Example: data to be processed, clock
  - **External data output**
    - Example: result data
  - **Status signals to the controller (to indicate what is occurring in the datapath)**
    - Example: when an adder has overflowed

Inputs and Outputs

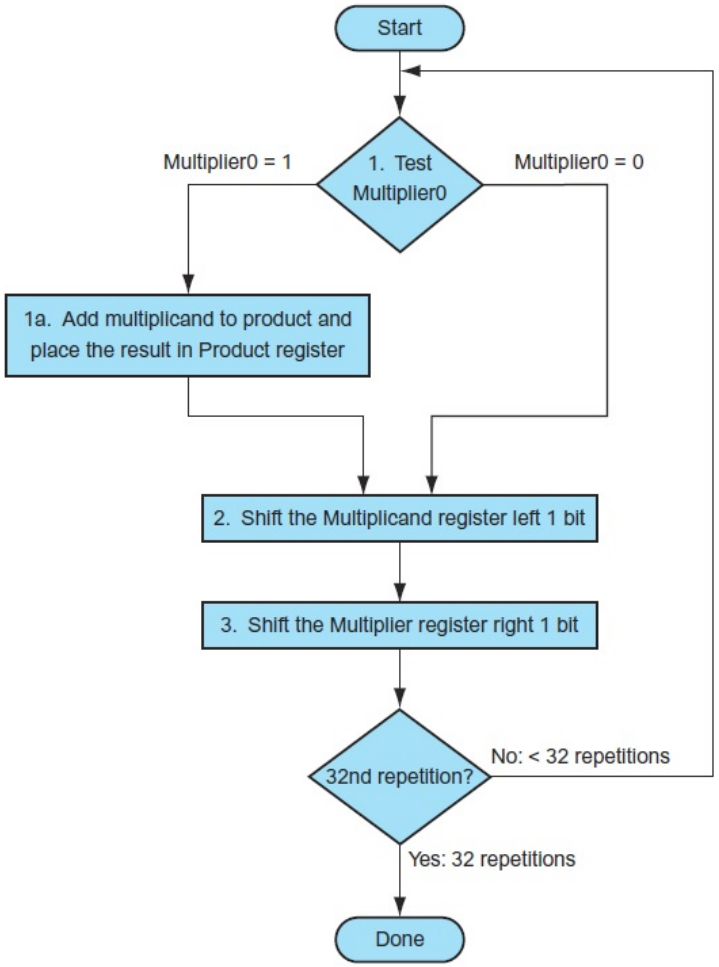


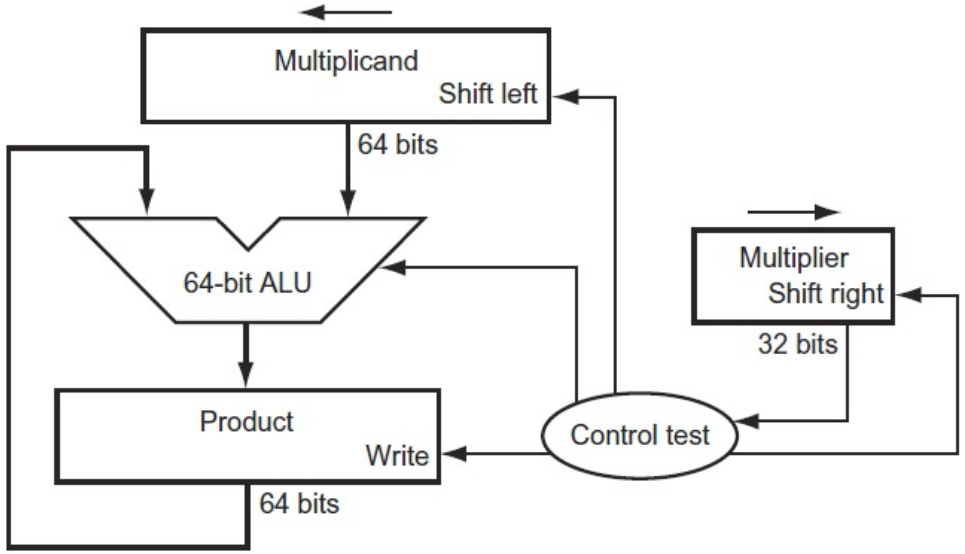
- Controller inputs and outputs
  - **External input**
    - Example: reset signal, mode select, clock
  - **External output**
    - Example: done flag to outside world
  - **Control signals to the datapath**
    - Example: mux select, enable signals

Complex Digital Design: ASM Design Steps

- Given a specification, to design a **complex digital system using ASMs**, the following steps are involved:
  1. Translate specification into **pseudocode**.
  2. Design a **datapath block diagram** based on high-level ASM. Also called an execution unit. (Some references decompose block diagram into a datapath block diagram and controller block diagram.)
  3. Draw **top-level interface diagram**. This diagram connects the datapath with the controller to show all inputs, outputs, and internal connections of the entire digital system.
  4. Design **detailed controller ASM** based on high-level ASM. Detailed means using the exact signal names, not pseudocode representations.
- **After this process you have three results:**
  - Datapath: represented by a **datapath block diagram**
  - Controller: represented by a **detailed controller ASM**
  - Top-Level: represented by **top-level interface diagram**
- **From this it is "easy" to translate into VHDL**

Complex Digital Design: ASM Design Steps





How to "easily" translate into VHDL

- Given 4 ASM design steps are complete, translate into VHDL
- 3 VHDL translation steps:
  1. Code **detailed controller FSM** using **detailed controller FSM**
  2. Code **datapath** using **datapath block diagram**
  3. Code **top-level** using **top-level interface diagram**

Coding Step 1: Detailed Controller FSM

- **We code the detailed controller FSM about the same as we would any Moore or Mealy FSM**
- **Use the top-level interface diagram to help with entity ports (inputs and outputs)**
- **Section 1: FSM Register**
  - Reset and clocking of next state
  - Same as described earlier for simple FSM
- **Section 2: Next state function**
  - Use the detailed controller ASM to create the next state function
  - Same as described earlier for simple FSM
- **Section 3: Output function**
  - This is slightly different than the simple FSM described earlier
  - **First, set all outputs to default: this saves on typing and eliminates potential errors**
    - **Most of the time the outputs of the FSM (which go to the controller or to the outside world) are the same value.**
    - **Only indicate when the FSM output value changes from the default.**

Coding Step 2: Datapath

- Code the datapath based on the datapath block diagram
  - May have to use structural coding style
  - First generate building blocks
- Use top-level interface diagram to help with entity input and output PORTS
- **All internal wires of the datapath block diagram are SIGNALS in datapath.vhd**

Step 3: Code top-level entity

- Code top-level entity using specification and top-level entity diagram
  - All external inputs and outputs to either datapath or controller should be top-level entity **PORTS**
  - All wires between the datapath and controller should be declared as **SIGNALS** in the top-level architecture
- This step should consist of
  - Declaring **SIGNALS** between datapath and controller
  - Declaring **components datapath and controller** (either in main code or package)
  - Instantiating **datapath.vhd and controller.vhd** and making proper port mapping